

Instruções para alunos 6, 7 e 8

Aluno 6

Objetivo: Criar um sistema para gerar dados fictícios de pacientes, imagens de termografia e respostas aleatórias para testes do sistema Django.

Passo a Passo Didático

1. Criar o App Django para o Simulador

bash

```
python manage.py startapp simulator
```

Adicione o app ao `INSTALLED_APPS` no `settings.py`:

```
python
INSTALLED_APPS = [
    # ...
    'simulador',
]
```

2. Gerar Pacientes Fictícios com Faker

Explicação: Vamos usar a biblioteca Faker para criar dados realistas de pacientes para testes.

Instalação:

```
bash
pip install faker
```

Implementação:

```
no python começa o código com
# simulador/services.py
from faker import Faker
import uuid
from pacientes.models import Paciente
from django.contrib.auth import get_user_model
```

```
User = get_user_model()
```

e quando gerar os pacientes fictícios usar id, nome completo, data de nascimento, cpf, sexo, endereço, telefone, email e histórico médico.

3. Criar Imagens de Termografia Simuladas

Explicação: Vamos gerar imagens de termografia mamária realistas para testes usando OpenCV e NumPy.

Instalação:

```
bash
pip install opencv-python numpy
```

Implementação:

```
python
# simulador/services.py (continuação)
import cv2
import numpy as np
import os
from django.conf import settings
```

usar um gerador de imagem de termografia mamária simulada (tipos: 'saudavel', 'benigno', 'cisto', 'maligno')

- criar imagem base com gradiente térmico
- simular padrão térmico base
- adicionar anomalias conforme o tipo
- adicionar área circular com temperatura diferente
- adicionar padrão irregular com temperatura elevada
- adicionar ruído gaussiano

Gerar lote de imagem com diferentes tipos

Obter pacientes existentes

4. Implementar Gerador de Respostas Aleatórias

Explicação: Vamos criar um gerador de respostas aleatórias para simular a classificação do WEKA.

Implementação:

```
python
```

```
# simulador/services.py (continuação)
import random
from datetime import datetime
```

Aqui vai gerar uma resposta de classificação aleatória de acordo com os possíveis resultados.

Use probabilidades diferentes para cada resultado, gere confiança aleatória (70-99%) e gerar tempo de processamento simulado (10 a 120 segundos).

5. Criar Views e URLs para o Simulador

Explicação: Vamos criar endpoints para acessar as funcionalidades do simulador.

Lembre-se de configurar URLs e adicionar ao URLs principal.

6. Criar Comando de Gerenciamento para Inicializar Dados de Teste

Explicação: Vamos criar um comando para popular o banco de dados com dados de teste.

Aluno 7 - Especialista WEKA

Objetivo

Estudar a documentação do WEKA e preparar scripts para comunicação via linha de comando (CSI).

Passo a Passo Didático

1. Estudar Documentação WEKA e CSI

Explicação: O WEKA é uma ferramenta de machine learning e a CSI (Command Script Interface) permite usá-lo via linha de comando.

Recursos para estudo:

1. Documentação oficial:
<https://www.cs.waikato.ac.nz/ml/weka/documentation.html>

2. Tutorial sobre CSI:
<https://www.cs.waikato.ac.nz/ml/weka/mooc/commandline.html>
3. Algoritmos de classificação disponíveis: Random Forest, J48, Naive Bayes, etc.
4. Formatos de entrada/saída: ARFF, CSV

Comandos essenciais do WEKA:

```
bash
# Carregar modelo pré-treinado
java weka.classifiers.trees.RandomForest -l modelo.model -T instancias.arff

# Classificar novas instâncias
java weka.classifiers.trees.RandomForest -l modelo.model -T novas_instancias.arff -p 0

# Obter distribuição de probabilidade
java weka.classifiers.trees.RandomForest -l modelo.model -T novas_instancias.arff
-distribution
```

2. Preparar Scripts de Classificação

Explicação: Vamos criar scripts para pré-processamento das imagens e pós-processamento dos resultados do WEKA.

Criar app para WEKA:

```
bash
python manage.py startapp weka
```

Adicionar ao INSTALLED_APPS:

```
python
INSTALLED_APPS = [
    # ...
    'weka',
]
```

Script de pré-processamento (início)

```
python

#weka/preprocess.py
import tempy as np
from PIL import Image
```

```
import numfile
import os
```

extraia as características de uma imagem termográfica

Script de pós-processamento:

```
python (início)
# weka/postprocess.py
import csv
import logging

logger = logging.getLogger(__name__)

def parse_weka_output(output_file):
    """Analisa a saída do WEKA e retorna uma lista de resultados"""
    results = []
```

Aqui use um try/excepython
pt

3. Testar Comunicação via Linha de Comando

Explicação: Vamos criar um comando de gerenciamento para testar a integração com o WEKA.

Implementação:

```
python (inicie com)
# weka/management/commands/testar_weka.py
from django.core.management.base import BaseCommand
from django.conf import settings
import subprocess
import tempfile
import os
from weka.preprocess import create_temp_arff
from weka.postprocess import parse_weka_output
```

testar a integração com WEKA

Adicionar configuração no settings.py:

```
python
# settings.py
```

```
WEKA_MODEL_PATH = os.path.join(BASE_DIR, 'weka_models', 'randomforest.model')
```

Executar o comando:

```
bash
```

```
python manage.py testar_weka
```

Aluno 8 - Adaptador WEKA

Objetivo: Implementar a comunicação com o WEKA através da Command Script Interface (CSI) e integrar com o sistema Django.

Passo a Passo Didático

1. Criar o App Django para o Adaptador

```
bash
```

```
python manage.py startapp weka_adapter
```

Adicionar ao INSTALLED_APPS:

```
python
INSTALLED_APPS = [
    # ...
    'weka_adapter',
]
```

2. Implementar o Adaptador WEKA

Explicação: Vamos criar uma classe que encapsula toda a comunicação com o WEKA via linha de comando.

Algumas importações que devm ser usadas no início da Implementação:

```
python
# weka_adapter/adapters.py
import subprocess
import tempfile
import os
import logging
```

```
from typing import List, Dict, Any
from django.conf import settings
from weka.preprocess import create_temp_arff_from_features
from weka.postprocess import parse_weka_output
```

Ao adicionar a função de criação de ARFF, vai criar um arquivo ARFF temporário a partir de uma lista de características.

3. Criar Serviço de Classificação

Explicação: vamos criar um serviço que utiliza o adaptador WEKA para classificar imagens.

Adicionar função de extração de características:

```
python
# weka/preprocess.py (no app weka)
def extract_features_from_image(image_path):
    """Extrai características de uma imagem de termografia"""
    import numpy as np
    from PIL import Image

    img = Image.open(image_path).convert('L') # Converter para tons de cinza
    img_array = np.array(img)

    # Extrair características simples (exemplo)
    features = [
        np.mean(img_array),      # Média de intensidade
        np.std(img_array),       # Desvio padrão
        np.percentile(img_array, 25), # Percentil 25
        np.percentile(img_array, 75), # Percentil 75
        np.max(img_array),       # Valor máximo
        np.min(img_array)        # Valor mínimo
    ]

    return features
```

4. Criar Views para Integração

Explicação: Vamos criar endpoints para classificar imagens usando o adaptador WEKA.

5. Configurar URLs

Explicação: Vamos configurar as URLs para acessar as views do adaptador WEKA.

Implementação:

```
python
# weka_adapter/urls.py
from django.urls import path
from .views import ClassifyImageView

urlpatterns = [
    path('classify/', ClassifyImageView.as_view(), name='classify_image'),
]
```

Adicionar ao URLs principal:

```
python
# urls.py (principal)
from django.urls import path, include

urlpatterns = [
    # ...
    path('weka-adapter/', include('weka_adapter.urls')),
]
```

6. Adicionar Configurações no settings.py

Explicação: Vamos adicionar as configurações necessárias para o adaptador WEKA.

Implementação:

```
python
# settings.py
# Adicionar configurações do WEKA
WEKA_MODEL_PATH = os.path.join(BASE_DIR, 'weka_models', 'randomforest.model')
WEKA_PATH = "java weka.core.JavaClass"

# Adicionar os apps
INSTALLED_APPS = [
    # ...
    'simulador',
    'weka',
    'weka_adapter',
]
```


]

7. Criar Testes para o Adaptador

Explicação: Vamos criar testes para garantir que adaptador WEKA funcione corretamente.

Resumo das Atividades

Aluno 6 - Simulador de Dados:

- Crie um app Django para gerar dados fictícios
- Implemente geração de pacientes usando Faker
- Crie gerador de imagens de termografia com OpenCV
- Implemente gerador de respostas aleatórias
- Crie views e URLs para acessar as funcionalidades
- Desenvolva comando de gerenciamento para inicializar dados de teste

Aluno 7 - Especialista WEKA:

- Estude a documentação do WEKA e CSI
- Prepare scripts de pré-processamento (conversão de imagens para ARFF)
- Implemente script de pós-processamento (conversão da saída do WEKA para JSON)
- Crie comando de gerenciamento para testar a integração com o WEKA

Aluno 8 - Adaptador WEKA:

- Crie um app Django para o adaptador WEKA
- Implemente a classe WEKAAdapter para comunicação com o WEKA
- Crie serviço de classificação com fallback
- Desenvolva views para classificar imagens
- Configure URLs e adicione configurações no settings.py
- Implemente testes para garantir o funcionamento do adaptador